Let's give it a whirl.

Before we start, though, you'll need to find a directory on your machine where you want to generate the Rails project files. As our upcoming Docker commands will affect our local files (we'll cover exactly how shortly), I recommend that you create a new, empty folder where you'll run the steps from. For example:

```
$ mkdir ~/docker_for_rails_developers
$ cd ~/docker_for_rails_developers
```

All set? Great. We're now going to start an interactive Bash shell inside a container based on the now familiar ruby:2.6 image:

```
$ docker run -i -t --rm -v ${PWD}:/usr/src/app ruby:2.6 bash
```

You can see we're using the --rm option to create a throwaway container that will be deleted once we are done with it. There are also some new options (-i, -t, and -v ${PWD}:/usr/src/app) that we haven't seen before. We'll come back to these in a moment. For now, though, when you run this command, you should be greeted by a terminal prompt that looks something like this:

```
root@0c286e8bda42:/#
```

This different prompt shows that we're now successfully running a Bash shell *inside* a container. The root@ and # indicate that we're the root user—this is the default user inside a container.

From this new Bash prompt, we now can issue any commands we want the container to run. This begs the question…what do we want to run? Remember: we're trying to generate a new Rails project. So, first, let's move into the folder we're going to use for our project:

```
root@0c286e8bda42:/# cd /usr/src/app
```

Now let's install the Rails gem:

```
root@0c286e8bda42:/usr/src/app# gem install rails
```

Rails Version



The examples in this book have been built and tested against Rails 5.2.2—the latest version at the time of writing. However, except where we use new Rails features, everything should largely work for previous versions of Rails too.

You should see the Rails gem and all its dependencies being installed. This means we're now ready to generate our project:

```
root@0c286e8bda42:/usr/src/app# rails new myapp --skip-test --skip-bundle
```